CAPSTONE PROJECT PROPOSAL

NTG-Agent vNext: Next-Generation Enterprise AI Agent Platform

Student Team	QUOTA
Students	Vu Gia An – s3926888 Nguyen Tien – s3924427 Tran Nguyen Quy Khang – s3929513 Nguyen Dinh Minh Chau – s3923010
Academic Supervisor	Tran Nhat Quang – RMIT Lecturer
Industry Supervisors	Nguyen Le Quynh Thien – Technical Manager Nguyen Sieu Anh – Technical Architect Doan Mai Hai Tuyen - HR
Company	NashTech Company

Table of Contents

List of Tables *3*

List of Figures *3*

Abstract *4*

1. Project Background & Problem Statement *4*

1.1 Literature Review **4**

1.1.1 Retrieval-Augmented Generation (RAG) 4

1.1.2 Multi-Agent Systems & Orchestration 5

1.1.3 Agent Memory Models 5

1.1.4 Model Context Protocol & Tool Integration 6

1.1.5 Evaluation & Observability for AI Agents 6

1.1.6 Summary of Literature Gaps 7

1.2 Existing Solutions Analysis **8**

1.3 Proposed Solution **9**

1.3.1 Solution Overview 9

1.3.2 Technology Stack 11

2. Scope of Work and Performance Evaluation *12*

2.1 Scope of Work **12**

2.1.1 In-Scope Deliverables 12

2.1.2 Out-of-Scope Items 12

2.2 Performance Evaluation **13**

2.2.1 Technical Performance KPIs 13

2.2.2 Code Quality & Security Metrics 13

2.2.3 Community & Business Metrics 13

2.2.4 Evaluation Methodology 14

3. Project Timeline, Resource Planning and Risk Management *14*

3.1 Project Timeline **14**

3.1.1 Overall Project Schedule 14

3.1.2 Detailed Sprint Breakdown 15

3.1.3 Key Milestones 16

3.2 Resource Planning **18**

3.3 Risk Management **19**

3.3.1 Risk Register 19

3.3.2 Risk Priority Matrix 22

References *24*

List of Tables

Table 1: Summarises The Comparison 8
Table 2: Estimated Cost 18
Table 3: Risk Summary Table 20
Table 4: Probability x Impact Matrix 22
Table 5: Priority Bands 23

List of Figures

Figure 1: Phase 0 – Discovery and Planning 15
Figure 2: Phase 1 – Discovery and Implement RAG 15
Figure 3: Phase 2 – Advanced Agents and Communication 15
Figure 4: Phase 3 – Production Hardening 16
Figure 5: Phase 4 – Polish, Evaluation and Community 16

Abstract

Enterprise adoption of Large Language Models is increasingly constrained not by raw model capability but by the surrounding agent platform: how reliably an agent can retrieve proprietary knowledge, remember user context, coordinate specialised sub-agents, integrate tools securely, and operate within enterprise observability and governance boundaries. The predecessor system, NTG-Agent v1, provides a sound .NET 10 microservices skeleton but operates as a single-agent assistant whose retrieval, memory, tooling, and security layers fall short of production requirements.

This capstone, undertaken by team QUOTA in partnership with NashTech, delivers **NTG-Agent vNext**: a next-generation, open-source enterprise AI agent platform built natively on .NET 10, .NET Aspire, Microsoft Semantic Kernel, and the Microsoft Agent Framework. Five advances drive the upgrade: a hybrid, citation-grounded RAG pipeline; a four-layer memory model; a multi-agent orchestration layer with an audit-logged Agent-to-Agent protocol; a streaming Blazor Agent-Generated UI; and a production-grade security, observability, and workflow stack.

Delivered across four phases over four months, the project will be evaluated against ≥ 85% RAG faithfulness, ≥ 90% multi-agent task completion, and SonarCloud quality-gate compliance, and is intended as a reusable .NET-native architectural template for NashTech's future AI solutions.

1. Project Background & Problem Statement

1.1 Literature Review

1.1.1 Retrieval-Augmented Generation (RAG)

While generative language models have transformed natural language processing, their use in enterprise environments is often hindered by limited access to proprietary or real-time data. Retrieval-Augmented Generation (RAG) addresses this by combining a retrieval mechanism with the generative model, enabling access to external knowledge bases at inference time [1]. The standard pipeline pre-processes documents into chunks, converts them into dense vector embeddings stored in a vector database, and at query time retrieves the top-k semantically similar chunks as grounding context [2]. The current literature distinguishes naive, advanced, and modular RAG architectures, with **Modular RAG** — the approach adopted here — decoupling indexing, retrieval, and generation into independently configurable components, which is essential for enterprise knowledge bases that change frequently [3]. In the proposed system, RAG is implemented as a dedicated microservice using **Microsoft Kernel Memory**, with the Orchestrator interacting through the **IKernelMemory** interface for ingestion and semantic search. Centralising retrieval in a single service maintains

a strict security boundary around proprietary data while still providing agents with the organisational context they need.

1.1.2 Multi-Agent Systems & Orchestration

While early enterprise AI systems relied on monolithic, single-agent paradigms, current literature identifies this approach as a bottleneck for complex, multi-step problem-solving, since user requests increasingly span diverse domains such as code generation and system integration. Researchers therefore advocate Multi-Agent Systems (MAS), in which specialised agents (e.g. triage, technical, and analytical) collaborate autonomously to resolve sophisticated workflows [4]. Robust MAS depends on intelligent routing, autonomous handoff protocols that let agents delegate sub-tasks when their domain limits are reached, and a shared context window that passes through the execution pipeline. Ensures seamless transitions without redundant prompting [5]. Architecturally, the field is moving from static linear handoffs toward dynamic, graph-based interactions, with hierarchical *Manager–Worker* orchestration emerging as a prominent pattern in which a primary orchestrator decomposes requests and dispatches tasks concurrently to specialised workers [6], while standardised integration layers such as the Model Context Protocol (MCP) keep agent logic modular and independent of the underlying infrastructure [7]. In applied system design, these principles enable dynamic workflow generation at runtime and significant token-cost optimization, allocating lightweight models for triage and reserving high-parameter reasoning models for complex execution, improving both efficiency and user transparency during automated *thinking* phases.

1.1.3 Agent Memory Models

Memory elevates a Large Language Model from a stateless generator into an agent capable of long-horizon, multi-session interaction; without it, the agent is bound by a fixed context window and cannot accumulate experience [8]. Drawing on the Atkinson–Shiffrin multi-store model, recent surveys of LLM-based agents adopt a taxonomy of working, episodic, semantic, and procedural memory [9]. Two architectures are particularly influential: MemGPT pages a fixed main context against an external store to give the agent an effectively unlimited working set [10], and Generative Agents records observations in a timestamped memory stream retrieved by recency, importance, and relevance [11]. Subsequent surveys consolidate these into fundamental operations, consolidation, updating, indexing, forgetting, retrieval, and compression, that any production memory system must implement [12][13]. In NTG-Agent v1, only a partial subset is implemented: the Orchestrator's **UserMemoryService** extracts persistent user facts and retrieves them by embedding similarity [14], which corresponds to a basic long-term semantic memory and lacks episodic summaries, cross-user semantic knowledge, and explicit working-memory budgeting. NTG-Agent vNext therefore adopts a four-layer model, short-term, long-term, episodic, and semantic, that reuses **UserMemoryService** as the long-term layer and adds dedicated stores for episodic summaries and cross-user semantic entities.

1.1.4 Model Context Protocol & Tool Integration

The ability of an LLM to invoke external tools is a prerequisite for grounded, action-capable agents. Early work such as Toolformer showed that language models can learn when to call APIs, what arguments to pass, and how to incorporate the returned results [15]. Provider-specific function-calling schemas followed, producing the M × N integration problem in which M models must be connected to N tools through bespoke adapters [16], [17]. The Model Context Protocol (MCP), introduced by Anthropic in November 2024 and inspired by the Language Server Protocol, addresses this fragmentation through a vendor-neutral, JSON-RPC 2.0 client–server interface that exposes three primitives — Resources (read-only data), Tools (executable functions), and Prompts (reusable templates) [16], [18], [19]. It has since been adopted by OpenAI, Google DeepMind, and Microsoft as the de facto interoperability standard [16], [17]. NTG-Agent v1 already participates in this ecosystem through **NTG.Agent.MCP.Server**, an ASP.NET Core service **using ModelContextProtocol.AspNetCore** and **MapMcp** routing, but exposes only two tools (**SearchOnlineTool** and a **MonkeyService** example) registered statically at startup, after which the Orchestrator's **AgentFactory** discovers and binds them to the agent kernel [18].

Recent studies show that MCP also enlarges the attack surface: tool descriptions, schemas, and server-provided context all enter the model's prompt and can be weaponised through indirect prompt injection, tool poisoning, and capability chaining [19], [20]. Empirical benchmarks such as MCPSecBench and MCPTox report attack success rates above 70% on unhardened agents, and OWASP has elevated prompt injection to the top of its Top 10 risks for LLM applications [20], [21]. Two limitations of the v1 implementation therefore motivate the vNext design. First, a two-tool catalogue is too narrow for realistic enterprise workflows and must be expanded with connectors such as GitHub, JIRA, and code execution. Second, static startup-time registration prevents administrators from on-boarding new MCP servers without redeploying the platform. NTG-Agent vNext therefore introduces a dynamic MCP tool registry exposed through the Admin UI and a security-aware execution layer (sandboxing, schema validation, authentication) aligned with the least-privilege guidance emerging in the MCP security literature [19], [20].

1.1.5 Evaluation & Observability for AI Agents

Evaluating AI agent systems introduces complexities that standard software testing isn't built to handle. Because these agents operate in open-ended, non-deterministic environments, we cannot verify "correctness" through simple input-output comparisons. Instead, a robust evaluation framework must address multiple dimensions, specifically: response quality, tool-use accuracy, reasoning fidelity, and operational costs.While standard NLP metrics like BLEU and ROUGE are widely used, they are insufficient for this project. They measure surface-level lexical overlap rather than semantic quality or actual task completion. Our approach prioritizes more appropriate methodologies, including human evaluation rubrics and reference-based semantic similarity. For our RAG pipelines, we will utilize frameworks like RAGAS to measure faithfulness—ensuring answers are grounded in context—and context precision.Observability is equally

critical for moving these agents into production. We require the ability to diagnose failures and identify performance bottlenecks at scale. By adopting the OpenTelemetry standard, we can provide a vendor-neutral collection of traces and logs via OTLP.

1.1.6 Summary of Literature Gaps

The literature reviewed in sections 1.1.1 to 1.1.5 highlights several persistent gaps in enterprise AI research—specifically regarding multi-agent integration, production-grade portability, and standardized observability. NTG-Agent vNext is engineered to address these deficiencies through the following architectural advancements:

1. Knowledge Consistency in Multi-Agent Pipelines

While Retrieval-Augmented Generation (RAG) has matured, current research often treats retrieval in isolation. In complex orchestration, multiple agents frequently query overlapping knowledge sources, leading to potential conflicts. We address this by implementing a centralized Knowledge Service. By utilizing a unified interface with tag-based filtering, the system ensures cross-agent consistency and enforces knowledge-level access control, which is often overlooked in academic frameworks.

2. Enterprise Portability and Production Readiness

Existing orchestration frameworks like AutoGen or LangGraph, while powerful, are often tightly coupled to specific providers or require significant overhead to implement enterprise requirements like authentication and auditing. NTG-Agent vNext prioritizes production readiness by utilizing a provider-agnostic .NET 10 architecture. By integrating ASP.NET Core Identity and EF Core persistence, the platform transitions multi-agent systems from research-oriented prototypes to portable, secure enterprise applications.

3. Persistent and Context-Aware Long-Term Memory (LTM)

Standard frameworks typically treat agent memory as session-scoped or stateless. However, enterprise environments require persistent, user-specific memory that survives across distinct sessions. Our proposed LTM subsystem utilizes configurable confidence thresholds and retrieval limits, allowing for selective and durable memory retrieval—a critical requirement for personalized enterprise assistants.

4. Standardized Tool Integration (MCP)

The fragmentation of tool integration standards remains a hurdle for scalability. By adopting the Model Context Protocol (MCP), NTG-Agent vNext implements a dedicated MCP Server. This ensures that tool discovery and invocation remain standards-compliant and extensible, allowing the core orchestration layer to remain decoupled from individual tool implementations.

4. Integrated Observability and Cost Governance

As identified in current literature, evaluation tooling for agents in production is still immature. This project contributes a practical solution by integrating OpenTelemetry with custom metrics (e.g., agent interaction count and response latency) and granular token-cost tracking. This provides the transparency necessary for real-world enterprise deployment and cost management.

1.2 Existing Solutions Analysis

The four external platforms reviewed above, including LangChain/LangGraph, AutoGen, CrewAI, and Semantic Kernel, each address one or two parts of the agent-platform problem well, but none of them simultaneously satisfies the four constraints that define NashTech's requirements: a native .NET 10 implementation, production-grade multi-agent orchestration, durable workflow execution, and built-in enterprise security and observability. Table 1 summarises the comparison.

Table 1: Summarises The Comparison

Dimension	LangChain/ LangGraph	AutoGen	CrewAI	Semantic Kernel	NTG-Agent v1
Language/ Stack	Python	Python	Python	.NET/ Python	.NET 10
Orchestration Model	Graph-based, dynamic	Conversation-based	Role-based, fixed	Plugin-based	Single-agent, static
Long-term Memory	Partial (via plugins)	None	None	Partial (user facts only)	Partial (user facts only)
RAG Support	Strong	Minimal	Minimal	Strong (Kernel Memory)	Implemented, but limited
Tool Integration	Extensive (LangChain tools)	Sandboxed execution	Role-scoped tools	MCP/ plugins	MCP (2 tools, static)

Security Controls	Minimal	Minimal	Minimal	Minimal	Minimal
Observability	Via plugins	Minimal	Minimal	Partial	Missing
Enterprise Readiness	Low	Low	Low	Medium	Medium
Multi-Agent Support	Yes (LangGraph)	Yes	Yes	Partial	Groundwork only

Three dimensions of divergence stand out:

- **Orchestration**. All reviewed frameworks either rely on static role assignments (CrewAI), conversation-driven flows (AutoGen), or require Python infrastructure (LangChain/LangGraph). None natively supports runtime-dynamic agent assembly within a .NET enterprise stack. NTG-Agent vNext addresses this by implementing a Semantic Kernel-based orchestrator capable of assembling agent workflows at runtime based on user intent [21][22].

- **Memory Intelligence**. No reviewed solution implements a full four-layer memory model (working, episodic, semantic, procedural). NTG-Agent v1's User Fact Store covers only basic semantic memory of user attributes. **vNext** closes this gap by introducing dedicated episodic and semantic stores alongside the existing long-term layer, aligned with the cognitive taxonomy established in the literature [8][9][10][11].

- **Operational Scalability.** All reviewed frameworks treat security, observability, and cost governance as secondary concerns. NTG-Agent vNext differentiates itself by embedding these as first-class architectural requirements: a dynamic MCP tool registry with sandboxing [19][20], OpenTelemetry-based distributed tracing, and granular token-cost tracking [32], capabilities absent from every framework reviewed above.

1.3 Proposed Solution

1.3.1 Solution Overview

NTG-Agent vNext evolves the foundational NTG-Agent v1 from a static, single-agent prototype into a production-ready, autonomous multi-agent ecosystem. Driven by NashTech’s expectation to showcase open-

source engineering on the modern Microsoft stack (.NET 10, .NET Aspire, and Semantic Kernel), this project delivers a highly scalable enterprise AI platform that transitions away from monolithic chatbot paradigms. To meet NashTech's stringent standards, vNext addresses the architectural bottlenecks of its predecessor. It relies on a dynamic multi-agent orchestration framework where specialized agents collaborate to solve complex workflows. This intelligence is supported by a comprehensive 4-layer cognitive memory model, a hybrid citation-grounded RAG pipeline, and an extensible tool registry utilizing the Model Context Protocol. Robust observability via OpenTelemetry ensures strict token-cost tracking, security, and performance monitoring. NTG-Agent vNext resolves current system limitations and serves as a definitive, reusable architecture template for NashTech's future AI-driven business solutions.

1. Hybrid, Citation-Grounded RAG Pipeline

The Modular RAG pipeline of Section 1.1.1 is extended in three ways. Dense embedding search is combined with BM25 keyword search using Reciprocal Rank Fusion, so exact-term queries, code identifiers, product names, ticket numbers, are no longer lost [23]. A cross-encoder re-ranker then discards low-relevance chunks before they reach the LLM context window, improving precision without retraining the embedding model [23]. Every retrieved passage is returned with a structured source citation that the WebClient renders alongside the answer, directly addressing the enterprise trust requirement. Together with replacing the hard-coded retrieval limit reported in Section 1.2, these changes constitute the largest retrieval-quality upgrade in vNext.

2. Four-Layer Cognitively Grounded Memory Model

The single user-fact store in UserMemoryService [14] is generalised into a four-layer model, short-term, long-term, episodic, and semantic, aligned with the cognitive taxonomy adopted by recent agent-memory surveys [8][12][9]. The existing service is retained as the long-term layer, preserving its MinimumConfidenceThreshold and MaxMemoriesToRetrieve controls. Episodic memory is added to capture conversation summaries with temporal metadata, inspired by the memory stream of Generative Agents [11]. Semantic memory becomes a shared cross-user entity store that integrates with the planned graph-based retrieval component (Section 2.1.1). Short-term memory is implemented as an explicit token-budgeted working set, addressing the absence of working-memory budgeting.

3. Multi-Agent Orchestration with Agent-to-Agent Protocol

The Manager–Worker pattern from Section 1.1.2 is realised using the Microsoft Agent Framework GroupChat API: a Planner decomposes the request, a Task Router dispatches sub-tasks to specialist agents (Research, Code, Summary), and an optional Critic validates the merged response. Two design choices distinguish vNext. First, an explicit Agent-to-Agent (A2A) message envelope from_agent_id, to_agent_id, task_type, payload, callback_url, trace_id carries each delegated task across service boundaries and is persisted to SQL Server so every delegation is auditable. Second, the standardisation rationale for A2A mirrors that of MCP [16]: a typed

envelope removes the need for point-to-point integrations and lets the Agent Registry evolve without changing call sites. Non-dependent sub-tasks run in parallel when the Planner detects no data dependency.

4. Streaming Agent-Generated UI (AG-UI / A2UI)

Rather than restricting agents to plain-text replies, the Blazor frontend introduces a real-time event channel, implemented over Server-Sent Events and SignalR through which agents can stream reasoning traces, tool-call status, and structured UI fragments (cards, tables, forms, action buttons) that are rendered natively using Blazor's DynamicComponent. This brings the platform into line with the emerging class of agent interfaces that surface intermediate reasoning to end users.

5. Production-Grade Security, Observability and Workflows

Tag-only access control is replaced with full Role-Based Access Control, OIDC/SSO, per-user API-key management with rotation, and an immutable audit trail. The existing OpenTelemetry pipeline [18] is extended with distributed tracing across multi-agent calls, per-LLM cost dashboards, and RAG-quality metrics. Long-running tasks are managed by a new Workflow service on the .NET Durable Task Framework, adding resumable pipelines, cron-scheduled jobs, and Human-in-the-Loop approval gates. Together these additions close the gap, identified in current MCP and agent-safety research, between research prototypes and enterprise-deployable systems [19][20].

1.3.2 Technology Stack

NTG-Agent vNext is built entirely on the Microsoft .NET ecosystem, which aligns with NashTech's goal of producing an open-source enterprise showcase on the modern Microsoft stack. The solution is managed by **.NET Aspire**, which handles service orchestration, health monitoring, and shared configuration across all projects through **NTG.Agent.ServiceDefaults**.

The backend is organized into five services that map directly to the repository structure: **NTG.Agent.Orchestrator** acts as the central API and agent execution engine; **NTG.Agent.Knowledge** handles document ingestion and semantic search via **Microsoft Kernel Memory**; **NTG.Agent.MCP.Server** exposes tools over the Model Context Protocol; and **NTG.Agent.Admin** and **NTG.Agent.WebClient** serve as the administrator portal and end-user chat interface respectively, both built with **Blazor**. A YARP reverse proxy sits between the WebClient and Orchestrator as a Backend for Frontend, forwarding authenticated requests.

Agent logic is driven by the **Microsoft Agent Framework** and **Semantic Kernel**, supporting multiple LLM providers — **Azure OpenAI**, **OpenAI**, **GitHub Models**, and **Google Gemini** — switchable through configuration without code changes. **SQL Server** with **Entity Framework Core** handles persistence for conversation history, agent settings, and the A2A audit log.

The CI pipeline runs on **GitHub Actions** with **SonarCloud** for static analysis, while **OpenTelemetry** exports traces and metrics via **OTLP** for runtime observability.

2. Scope of Work and Performance Evaluation

2.1 Scope of Work

2.1.1 In-Scope Deliverables

The main goal of this project is to take the current NTG-Agent and turn it into a high-level, multi-agent platform using **.NET 10**. The primary deliverables include:

- **Advanced RAG (Retrieval-Augmented Generation):** moving beyond basic search by adding **Hybrid Search** (mixing semantic and keyword search) and **GraphRAG** to help the AI handle complex questions better.
- **A Smarter 4-Layer Memory System:** Instead of just remembering a few facts, the system will have **Short-Term**, **Long-Term**, **Episodic** (summaries of past chats), and **Semantic** (shared knowledge) memory layers.
- **Multi-Agent Coordination:** Using the **Microsoft Agent Framework**, we're building a "team" setup where a **Planner Agent** breaks down tasks and hands them off to specialized **Worker Agents**.
- **Agent-to-Agent (A2A) Messaging:** A new protocol so different agents can actually talk to each other and delegate work across services.
- **Dynamic "AG-UI":** Instead of just getting text back, the UI will show the agent's **"thinking steps"** in real-time and can even pop up interactive Blazor components like charts or forms.

2.1.2 Out-of-Scope Items

While the project is extensive, the following items are explicitly excluded or not mentioned as part of the vNext delivery:

- **Physical Infrastructure Hosting:** The proposal covers the software architecture and "production-grade infrastructure" patterns (via Microsoft Aspire), but does not include the physical provisioning or cost management of cloud hosting environments (Azure/AWS/GCP).
- **Custom LLM Training:** The project utilizes existing LLM providers (GPT-4, Gemini, Azure OpenAI); it does not include training or fine-tuning proprietary foundational models.
- **Client-Side Mobile Applications:** Deliverables focus on a Blazor-based WebClient and Admin portal; native mobile applications (iOS/Android) are not listed in the architecture.

2.2 Performance Evaluation

2.2.1 Technical Performance KPIs

To rigorously evaluate the system's performance and ensure it meets enterprise-grade standards, the technical key performance indicators (KPIs) are divided into two main categories: **System & Infrastructure Metrics** (focusing on latency, availability, and observability) and **AI Orchestration Metrics** (focusing on agent accuracy and token efficiency). The evaluation will rely on standard load-testing tools and OpenTelemetry logs.

2.2.2 Code Quality & Security Metrics

NTG-Agent vNext continues to use SonarCloud for enforcing code quality, which is already integrated with the project's GitHub CI.

2.2.3 Community & Business Metrics

Since NTG-Agent vNext is an open-source initiative, its success is measured by how well it grows within the developer community and its overall value as a NashTech engineering showcase. We are tracking the following indicators to measure this impact:

Community Metrics

- **CONTRIBUTING.md:** Providing clear guidelines for new developers.
- **Issue Templates:** Standardized forms for reporting bugs and requesting features.
- **GitHub Discussions:** A dedicated space for community Q&A and support.
- **Versioned Releases:** Regular updates accompanied by detailed changelogs.
- **DeepWiki Documentation:** Continuously updated technical guides.
- **Architecture Decision Records (ADR):** Documented rationales for all major design choices.

Business Metrics

- **Feature Completion:** The project aims to deliver at least **20 out of the 36 planned features**.
- **Technical Performance:** We are targeting a **RAG retrieval accuracy of ≥ 85%** (faithfulness score) and a **multi-agent task completion rate of ≥ 90%**.
- **Latency Targets:** System performance is set at **≤ 5s** for single agents and **≤ 15s** for multi-agent workflows.

- **Cost Management:** We are implementing granular **token-cost tracking** per agent and user to monitor and govern LLM expenses.

2.2.4 Evaluation Methodology

To validate the system's targets in Sections 2.2.1 and 2.2.3, we conduct two evaluations: offline benchmarking before release and continuous monitoring after deployment. For offline evaluation, we create a test set of representative enterprise requests and run them through the full system. RAG quality is measured using RAGAS, specifically the faithfulness score, which checks if the answer is supported by the retrieved information, with a target of ≥ 85%. Multi-agent scenarios are scored based on whether the complete task is resolved correctly end-to-end, targeting ≥ 90% completion. When automated metrics fall short, such as judging coherence or reasoning, a secondary LLM is used as an automated evaluator to catch regressions in the CI pipeline without manual review on every commit. On the production side, OpenTelemetry traces provide visibility into request durations across service and agent boundaries, compared against the ≤ 5s and ≤ 15s latency targets for single-agent and multi-agent workflows, respectively. Token usage and estimated cost are recorded per invocation to detect unexpected increases early. SonarCloud enforces code quality as part of every pull request, as described in Section 2.2.2. These two tracks ensure continuous measurement throughout the project, not just testing before release.

3. Project Timeline, Resource Planning and Risk Management

3.1 Project Timeline

3.1.1 Overall Project Schedule

The project is planned over four progressive phases spanning approximately four months, with each phase building on the previous one to gradually enhance system capability and readiness. The initial phase focuses on establishing core functionalities and system foundations, followed by a second phase that expands agent intelligence and inter-agent communication. The third phase emphasizes production readiness, including scalability, security, and advanced knowledge representation. Finally, the last phase is dedicated to refinement, optimization, and improving usability, along with preparing documentation for long-term maintenance and potential community contribution.

3.1.2 Detailed Sprint Breakdown

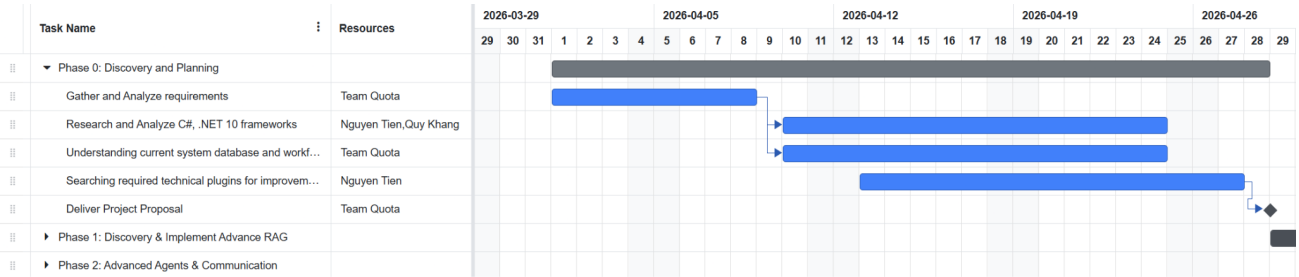

Figure 1: Phase 0 – Discovery and Planning

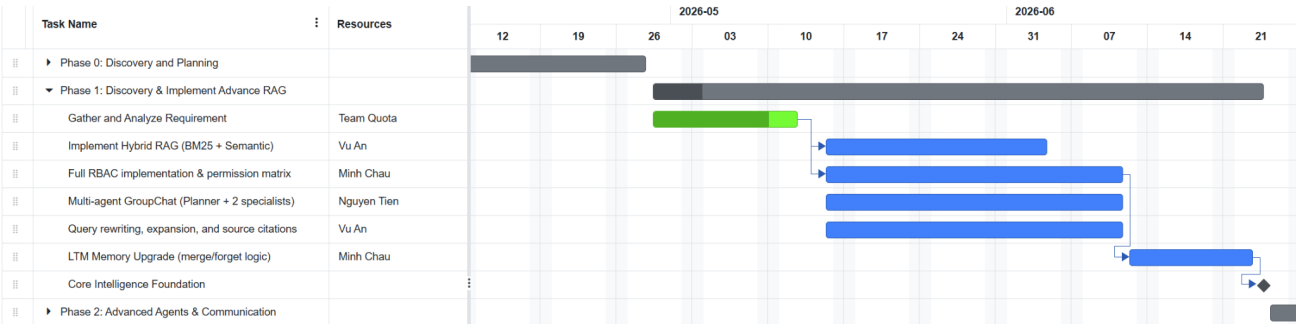

Figure 2: Phase 1 – Discovery and Implement RAG

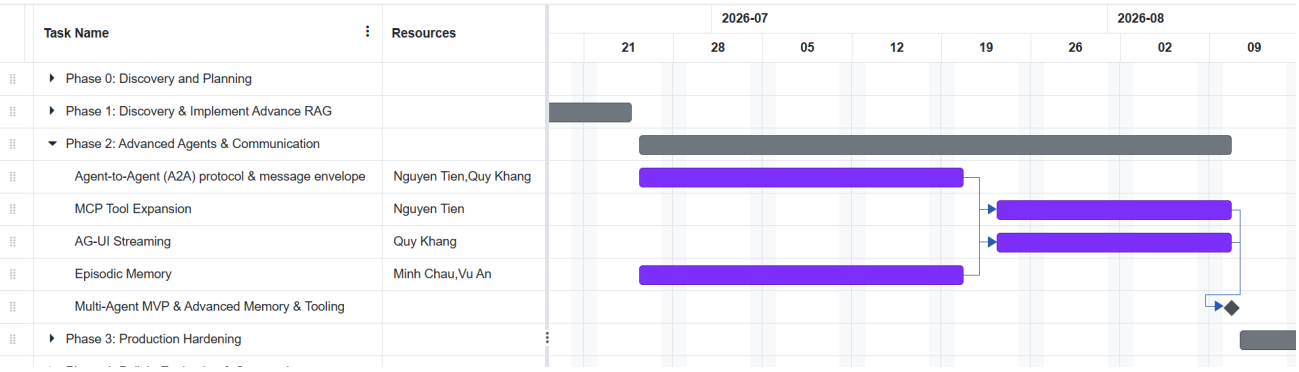

Figure 3: Phase 2 – Advanced Agents and Communication

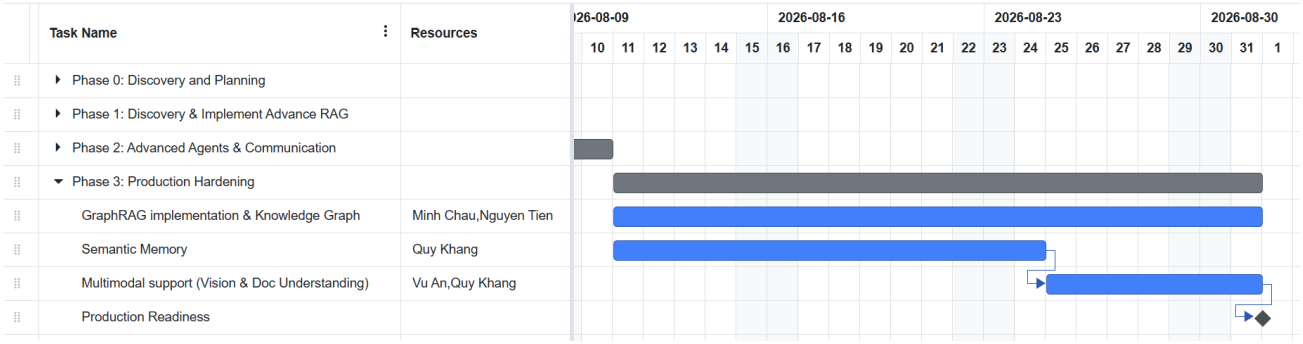

Figure 4: Phase 3 – Production Hardening

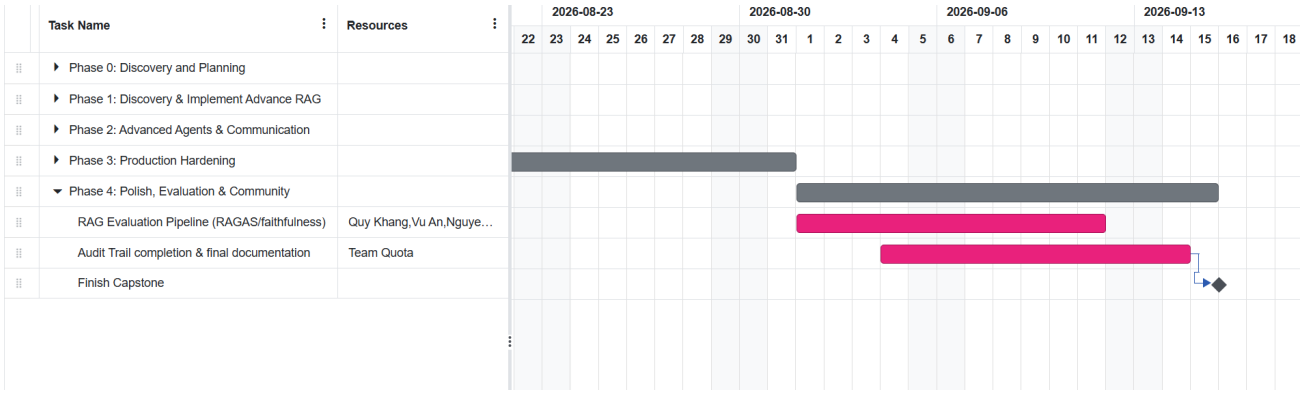

Figure 5: Phase 4 – Polish, Evaluation and Community

3.1.3 Key Milestones

1. Phase 0: Discovery and Planning (Deliver Project Proposal)

   Goal: Successfully grab the system architecture and deliver project proposals.

   Deliverables:

   - Understanding C#, .NET 10 framework and all the technical stack required for the project.
   - Propose an improvement roadmap for the current NTG-Agent system.
   - **Success Criteria:** Deliver a compose project proposal.

2. Phase 1: Discovery & Implement Advance RAG (Core Intelligence Foundation)

   Goal: Successfully upgrade the RAG pipeline and Security layer.

   Deliverables:

   - Hybrid RAG (Semantic + BM25) is fully operational.

- Role-Based Access Control (RBAC) is integrated into the Orchestrator.
- **Success Criteria:** The agent can retrieve internal documents with at least 80% accuracy while respecting user permissions.

3. Phase 2: Advanced Agents & Communication (Multi-Agent MVP & Advanced Memory & Tooling)

Goal: Transition from a single-agent system to a collaborative team. Enable the system to "remember" long-term and interact with external apps.

Deliverables:

- Planner/Router Agent can delegate tasks to at least two specialist agents.
- Agent-to-Agent (A2A) communication protocol is functional.
- MCP Tool Expansion (GitHub and JIRA connectors live).
- **Success Criteria:** A user can ask a complex question (e.g., "Analyze this code and search for documentation") and see the task split between two specialized agents.The agent can recall a decision made in a previous conversation

4. Phase 3: Production Hardening(Production Readiness)

Goal: Implement high-end features and enterprise-grade reliability.
Deliverables:

- GraphRAG implementation completed for complex relationship mapping.
- Durable Workflows active (tasks survive service restarts).
- OIDC/SSO (Single Sign-On) integrated.
- **Success Criteria:** The system can handle long-running background tasks and map connections across a large document library.

5. Phase 4: Polish, Evaluation & Community

Goal: Validation of metrics and project closure.
Deliverables:

- Full RAGAS evaluation report (Faithfulness/Relevance).
- Completion of the Admin Dashboard for visual workflow monitoring.
- Final documentation and "DeepWiki" update.
- **Success Criteria:** The project meets the "Community Metrics" and passes all SonarCloud quality gates with 0 vulnerabilities.

3.2 Resource Planning

To execute the NTG-Agent vNext project successfully, our team needs technical, physical, and financial resources. NashTech, our industry sponsor, will provide most of the foundational resources, reducing overhead and allowing the team to focus on architecture and development. Below is a detailed breakdown of the provided resources, their project lifecycle purposes, and associated costs:

Table 2: Estimated Cost

Category	Resource Item	Purpose	Estimated Cost
Technical resources	Source Code Access	Essential for understanding the existing architectural baseline, identifying limitations, and developing the vNext features	Provided by Nashtech (free)
Initial Setup	Setup instructions	Necessary to quickly replicate NashTech's development environment locally, minimizing setup time and preventing configuration discrepancies during deployment.	Provided by Nashtech (free)
Workspace & Hardware	On-site Office Workspace (20 hours/week commitment)	Facilitates intensive pair programming, real-time debugging, and direct interaction with industry supervisors. The professional	Provided by Nashtech (free)

		environment ensures high productivity.	
Communication & Collaboration	Dedicated Microsoft Teams Channel	Serves as the centralized hub for asynchronous communication, daily updates, file sharing, and quick Q&A sessions with NashTech mentors.	Provided by Nashtech (Enterprise License)
Knowledge & Training	Technical Training Meetings (Copilot, C#, MS Stack)	Accelerates the team's onboarding process. Ensures all members are aligned with enterprise-grade coding standards and proficient in utilizing modern AI development tools.	Provided by Nashtech (time investment)
Financial Support	Transportation Allowance	Subsidizes the commuting costs for team members to travel to the NashTech office, supporting the required 20-hour weekly on-site commitment.	500K VND/ member/month

3.3 Risk Management

3.3.1 Risk Register

The register is initialised from the architectural risks and extended with project-management, security, and compliance risks specific to the team’s context. It is intended as a living document: it will be reviewed at every sprint retrospective and updated whenever a risk materialises, is closed, or a new risk is identified.

Table 3: Risk Summary Table

ID	Category	Risk Description	Prob.	Impact	Score	Mitigation Plan
R-01	Technical / Dependency	Microsoft Agent Framework introduces breaking API changes during the project, forcing rework on the multi-agent orchestration layer.	Medium	High	6	Pin to a verified stable NuGet version; run weekly automated dependency scans; isolate framework calls behind an internal abstraction layer (IAgentFactory) so that upgrade impact is localised.
R-02	Technical / Complexity	Multi-agent orchestration produces unpredictable failure modes (deadlocks, infinite delegation loops, conflicting outputs).	High	High	9	Begin with a two-agent (Planner + one Specialist) baseline; introduce a hard maximum-step limit and a Critic-agent veto; expand the agent team only after the simpler scenario stabilises.
R-03	Cost / Operational	LLM token consumption exceeds the project budget, especially during multi-agent and evaluation runs.	Medium	Medium	4	Extend the existing token-tracking service to enforce a per-conversation token budget; configure cost-spike alerts in the Aspire Dashboard; default development environments to free-tier GitHub Models.
R-04	Technical / Scope	GraphRAG implementation proves too complex to deliver within the planned timeline.	High	Medium	6	Schedule GraphRAG into Phase 3 and treat it as a stretch goal; prefer an existing managed graph store (Neo4j or Azure Cosmos DB for Graph) over a custom implementation.
R-05	Schedule / Integration	OIDC/SSO integration with enterprise identity providers (Azure AD, Okta) consumes	Medium	Low	2	Use the well-established ASP.NET Core OIDC middleware as the single

ID	Category	Risk Description	Prob.	Impact	Score	Mitigation Plan
		more time than planned and delays Phase 3.				integration point; deliver Azure AD first as the reference, defer Okta and Google Workspace to a follow-on sprint.
R-06	Security	Newly added MCP tools — especially the code-execution sandbox — are exploited to run arbitrary code, exfiltrate secrets, or perform indirect prompt injection.	High	High	9	Enable Anthropic-style explicit user consent before any new tool is used; integrate an MCP-aware vulnerability scanner in CI.
R-07	Quality / Data	Memory extraction stores low-quality or incorrect facts about users, leading to personalised but wrong responses.	Medium	Medium	4	Tune extraction prompts and the MinimumConfidenceThreshold; add a per-memory user-facing edit/delete control in the Memory Dashboard; track precision in the RAG/Memory evaluation pipeline.
R-08	Quality / RAG	Hybrid RAG pipeline introduces regressions in answer quality compared to the existing semantic-only baseline.	Medium	Medium	4	Maintain the v1 semantic-only path as a feature-flagged fallback; require RAGAS evaluation to pass before promoting the hybrid path to default; capture retrieval traces in OpenTelemetry for diagnosis.
R-09	Resource / Team	Loss of a key contributor (illness, academic load, departure) delays critical-path features.	Medium	Medium	4	Maintain pair-ownership for every High-priority feature; enforce code review by at least one non-author; keep documentation to make hand-over is possible.

ID	Category	Risk Description	Prob.	Impact	Score	Mitigation Plan
R-10	Open-source / Adoption	vNext community uptake (stars, external pull requests) does not reach the targets stated in the success metrics.	Medium	Low	2	Maintain CONTRIBUTING.md and DeepWiki documentation; publish technical write-ups for major features; provide good-first-issue labels and prompt PR reviews; treat shortfalls as informational rather than blocking.

3.3.2 Risk Priority Matrix

Table 4: Probability x Impact Matrix

Probability ↓ / Impact →	Low (1)	Medium (2)	High (3)
High (3)	—	R-04	R-02, R-06
Medium (2)	R-05, R-10	R-03, R-07, R-08, R-09	R-01
Low (1)	—	—	—

Table 5: Priority Bands

Score range	Priority band	Required treatment	Review cadence
6 – 9	Critical	Active mitigation; explicit fallback plan; sign-off from project lead before sprint close.	Weekly
3 – 4	High	Active mitigation; trigger monitored; documented contingency.	Sprintly
2	Moderate	Mitigation planned; trigger monitored.	Bi-weekly
1	Low	Accept and monitor.	Monthly

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 9459–9471, 2020.

[2] K. Shuster et al., "Retrieval Augmentation Reduces Hallucination in Conversation," arXiv preprint arXiv:2104.07567, 2021.

[3] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2024.

[4] Microsoft, "Single agent vs. multiple agents," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ai-agents/single-agent-multiple-agents

[5] Microsoft, "Agent handoff," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/agent-framework/workflows/orchestrations/handoff?pivots=programming-language-csharp

[6] "Manager-worker communication patterns," Level Up – GitConnected. [Online]. Available: https://levelup.gitconnected.com/manager-worker-communication-patterns-c3580b9db5db

[7] Anthropic, "Introduction," Model Context Protocol. [Online]. Available: https://modelcontextprotocol.io/docs/getting-started/intro

[8] Z. Zhang et al., "A Survey on the Memory Mechanism of Large Language Model-based Agents," arXiv preprint arXiv:2404.13501, 2024.

[9] H. Liu et al., "From Human Memory to AI Memory: A Survey on Memory Mechanisms in the Era of LLMs," arXiv preprint, 2024.

[10] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560, 2023.

[11] J. S. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," in Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST), 2023, pp. 1–22.

[12] Y. Wu, Y. Liang, Y. Zhang, et al., "Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers," 2025, *arXiv:2603.07670*. [Online]. Available: https://arxiv.org/abs/2603.07670

[13] R. Wang et al., "Rethinking Memory in LLM-based Agents: Representations, Operations, and Emerging Topics," arXiv preprint, 2024.

[14] NashTech Garage, "NTG.Agent.Orchestrator," GitHub, 2026. [Online]. Available: https://github.com/nashtech-garage/ntg-agent

[15] T. Schick et al., "Toolformer: Language Models Can Teach Themselves to Use Tools," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, 2023.

[16] Anthropic, "Introducing the Model Context Protocol," Anthropic Blog, Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[17] Anthropic, "Model Context Protocol Specification," 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[18] NashTech Garage, "NTG-Agent source code and DeepWiki documentation," 2026. [Online]. Available: https://github.com/nashtech-garage/ntg-agent; https://deepwiki.com/nashtech-garage/ntg-agent. Accessed: Apr. 9, 2026.

[19] "MCPTox: A Benchmark for Tool Poisoning Attack on Real-World MCP Servers," arXiv preprint, 2025.

[20] "MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols," arXiv preprint, 2025.

[21] OWASP, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[21] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155, 2023.

[22] L. Wang et al., "A Survey on Large Language Model based Autonomous Agents," Front. Comput. Sci., vol. 18, no. 6, 2024.

[23] Y. Gao, Y. Xiong, X. Gao, et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," 2024, *arXiv:2312.10997*. [Online]. Available: https://arxiv.org/abs/2312.10997